

VestaFuture

Guest feedback into an experience package you can actually deliver · World Bank Small AI for Development Hackathon 2026, tourism · Mariyan Silva, Sri Lanka, solo entry

1. Challenge tackled

Noor runs coffee farm visits in the Sri Lankan hill country. Around seven guests a month find her by word of mouth. They leave happy and she never learns what they valued or what they wished she offered. Her feedback already exists, scattered across Google reviews, WhatsApp messages and remarks made at the gate that nobody wrote down, in four languages. Reading across all of it to find a pattern is analytical work she has no time, tooling or connectivity to do.

VestaFuture does that work on her own phone with no connection, and turns it into three experience packages ranked by cost. Every element traces back to the reviews that justify it and to a farm asset she has already told the app she has. The user is the operator, not the tourist.

2. Tools and models used

Xenova/paraphrase-multilingual-MiniLM-L12-v2	sentence embeddings, quantised, about 118 MB, on device
Transformers.js	runs the model in the browser, cached after first load
Web Worker	keeps embedding off the interface thread
Service worker and localStorage	offline caching and on device persistence
Rule based parser and templates	blob parsing, privacy stripping, package assembly. No generative text
Plain HTML, CSS, JavaScript	no framework, no build step, no backend, no account
Claude Code	build assistance. The author is not a developer

3. What worked well

- **It runs with the network off.** The model downloads once, then the full pipeline works in aeroplane mode. Verified in Chrome with the network disabled.
- **Messy pastes are handled.** A Google Reviews blob of 30 lines yielded 6 usable reviews. Reviewer names, relative dates, star words, owner replies and duplicate translation originals were discarded, a phone number was stripped, and the user confirmed the result before anything was saved.
- **Packages cannot be invented.** Assembly is deterministic, matching a theme against her stated capabilities and filling fixed templates. The tool cannot propose a farm asset she does not have.
- **Refusals are visible.** Themes below three supporting reviews are shown but never used. Where a package cannot be built, the app names what is missing, such as a theme one review short and a capability still unmarked, rather than failing silently.

4. What was challenging

- **Sinhala, the main finding.** Free clustering grouped Sinhala reviews with each other rather than by meaning. Three unrelated Sinhala reviews about roasting, tasting and picking formed a cluster large enough to pass the three review threshold, so the tool would have reported a theme that did not exist. Xenova/multilingual-e5-small was tested as a replacement. It understood Sinhala better but the language clustering persisted and its scores were worse on the other languages. The app now saves Sinhala and Tamil reviews, excludes them from counting, and explains why on screen.
- **Clustering in general.** Free clustering was replaced with similarity against a fixed set of readable theme descriptions. Less elegant, far more checkable, and stable across runs.
- **Synthetic data honestly labelled.** No real dataset exists for this and scraping real reviews was ruled out on privacy grounds. 30 synthetic reviews were generated and flagged through the whole pipeline, so evidence lines read “9 reviews support this, 7 of them examples” rather than hiding the provenance.
- **Not caching embeddings.** Reviews are re-read on every open, about 15 seconds on a laptop and longer on a cheap phone. The fix is known and was left out of the build window deliberately rather than risking storage changes late.

5. How the time was spent

0 to 2h	Sector brief, problem selection, scoping against the offline and small model constraints
2 to 5h	App structure and the capability profile screen
5 to 7h	Feedback capture, synthetic review generation
7 to 10h	Embedding model, theme matching, thresholds, offline verification
10 to 14h	Messy paste parser, privacy stripping at ingest, three cost tiers
14 to 16h	Sinhala testing, the alternative model, the negative result and its handling
16 to 18h	Dead end handling, interface fixes, videos, submission

Takeaway. A model that covers fifty languages does not serve fifty languages equally, and you only find that out by testing in your own. The failure was invisible until someone who reads Sinhala went looking for it.

Code and demo github.com/mhansinis/noor · mhansinis.github.io/noor